Unit 2

Requirement Engineering (RE) is a systematic process used to **identify, analyze, document, validate, and manage software requirements** throughout the project lifecycle.

---

1. Inception

Inception is the **starting phase** of requirement engineering.

Purpose:

- Identify stakeholders
- Understand the problem and business goals
- Define system scope and feasibility

Activities:

- Stakeholder identification
- Problem definition
- Initial system description

---

2. Requirement Elicitation

Elicitation is the process of **gathering requirements** from stakeholders.

Techniques:

- Interviews
- Questionnaires
- Workshops
- Observation
- Brainstorming
- Use cases

Outcome:

- Collection of functional and non-functional requirements

---

3. Elaboration

Elaboration involves **analyzing and refining requirements** to improve clarity.

Activities:

- Classify and organize requirements
- Create models (UML, DFD, use cases)
- Identify inconsistencies and missing requirements

---

4. Negotiation

Negotiation resolves **conflicts among stakeholders' requirements**.

Purpose:

- Prioritize requirements
- Balance cost, time, and resources
- Reach agreement on final requirements

Result:

- Approved and prioritized requirement set

---

5. Requirement Specification

Specification is the process of **documenting requirements clearly and formally**.

Document:

- **SRS (Software Requirement Specification)**

Contents:

- Functional requirements
- Non-functional requirements
- Interfaces and constraints

---

6. Requirement Validation

Validation ensures that requirements are **correct, complete, consistent, and feasible**.

Techniques:

- Reviews and inspections
- Prototyping
- Test-case derivation
- Stakeholder approval

---

7. Requirement Management

Requirement Management handles **changes to requirements throughout the project**.

Activities:

- Change control
- Requirement traceability
- Version management
- Impact analysis

Purpose:

- Ensure requirements remain up-to-date
- Control scope creep

Process Flow

Inception → Elicitation → Elaboration → Negotiation → Specification → Validation → Requirement Management

In short

Requirements Engineering: Seven Distinct Tasks

Requirements Engineering (RE) is a structured process used to identify, analyze, document, validate, and manage software requirements.
It consists of the following **seven distinct tasks**:

---

1. Inception

- Identifies stakeholders and system goals
- Defines problem scope and feasibility
- Establishes business objectives

---

2. Requirement Gathering / Elicitation

- Gathers requirements from stakeholders
- Uses interviews, questionnaires, workshops, observation, etc.
- Collects functional and non-functional requirements

---

3. Elaboration

- Analyzes and refines gathered requirements
- Develops models such as use cases, UML, DFD
- Removes ambiguity and inconsistencies

---

4. Negotiation

- Resolves conflicts between stakeholder requirements
- Prioritizes requirements
- Balances cost, time, and resources

---

5. Specification

- Documents requirements clearly and formally
- Produces the **Software Requirement Specification (SRS)**
- Acts as a reference for design and development

---

6. Validation

- Ensures requirements are correct, complete, consistent, and feasible
- Uses reviews, prototyping, and test cases
- Confirms stakeholder approval

---

7. Requirements Management

- Manages requirement changes
- Maintains traceability
- Controls scope creep throughout the project

Requirement Modeling

What is Requirement Modeling?

Requirement Modeling is the process of **representing system requirements visually or logically** so they are **easy to understand, analyze, and validate**.

It helps stakeholders and developers **clearly see how the system will work**.

Objectives of Requirement Modeling

- Understand system behavior clearly
- Remove ambiguity and confusion
- Identify missing or conflicting requirements
- Improve communication between users and developers

Types of Requirement Models

1. **Use Case Model**
    - Shows interaction between **users (actors)** and the system
    - Example: Login, Register, Place Order
2. **Data Flow Diagram (DFD)**
    - Shows **flow of data** in the system
    - Includes processes, data stores, inputs, outputs
3. **Entity Relationship Diagram (ERD)**
    - Represents **data entities and relationships**
    - Example: User, Order, Product
4. **State Diagram**
    - Shows **different states of a system**
    - Example: Order → Placed → Shipped → Delivered
5. **Activity Diagram**
    - Represents **workflow of activities**

Importance of Requirement Modeling

- Makes requirements **easy to visualize**
- Helps in early error detection
- Supports better system design
- Improves requirement validation

Requirement Modeling

Requirement Modeling is the process of representing system requirements in a structured and visual form so that both developers and users can clearly understand the system.

In simple words: Converting requirements into diagrams and models.

Main Techniques

1. Scenario-Based Modeling

It describes how the user will interact with the system in real situations (Use Cases).

Example (Smart Home):
User → Login → Select Light → Press ON → Light turns ON

Tools:

- Use Case Diagram
- Use Case Description

---

2. Data Modeling

It defines what data will exist in the system and how different data elements are related.

Example:
User, Device, Sensor Data, Room

Tool:

- ER Diagram (Entity-Relationship Diagram)

---

3. Flow-Oriented Modeling

It shows how data moves through the system.

Tool:

- Data Flow Diagram (DFD)

---

4. Class-Based Modeling

It represents the object-oriented structure of the system.

Tool:

- UML Class Diagram

---

5. Behavioral Modeling

It shows how the system behaves under different conditions.

Tools:

- State Diagram
- Sequence Diagram

Requirement Specification

Requirement Specification is the process of writing all system requirements clearly in a formal document called the **SRS (Software Requirement Specification)**.

In simple words: Writing requirements in a structured document.

Contents of SRS

1. Introduction

- Purpose
- Scope
- Definitions

2. Overall Description

- System overview
- User characteristics
- Assumptions

3. Functional Requirements

What the system should do.

Example:

- User can log in
- User can turn light ON/OFF
- System shows temperature

---

4. Non-Functional Requirements

Quality attributes of the system.

- Performance
- Security
- Reliability
- Usability

---

5. External Interface Requirements

- User Interface
- Hardware Interface
- Software Interface

---

6. System Models / Diagrams

- Use Case Diagram
- DFD
- ER Diagram

Difference (Important for Exams)

Requirement Modeling	Requirement Specification
Visual representation (diagrams)	Written document (SRS)
Helps understand system	Helps implement system
Uses Use Case, DFD, ER, UML	Structured requirement document

Requirement Specification

What is Requirement Specification?

Requirement Specification is the process of **documenting all system requirements clearly and formally** in a structured way.

The output of this process is the **SRS (Software Requirement Specification)** document.

Characteristics of Good Requirement Specification

A good requirement specification should be:

- Clear and unambiguous
- Complete
- Consistent
- Verifiable
- Traceable

Contents of Requirement Specification

1. **Functional Requirements**
    - What the system should do
    - Example: “The system shall allow users to log in.”
2. **Non-Functional Requirements**
    - Performance, security, reliability, usability
    - Example: “The system shall respond within 2 seconds.”
3. **Interface Requirements**
    - User interface
    - Hardware and software interfaces
4. **Constraints**
    - Technology, budget, time limitations
5. **Assumptions and Dependencies**
    - Conditions assumed during development

Importance of Requirement Specification

- Serves as a **legal agreement** between client and developer
- Acts as a **base for design, coding, and testing**
- Reduces development cost and rework
- Helps in project planning and estimation

Difference (Quick View)

Requirement Modelling	Requirement Specification
Visual representation	Written documentation
Uses diagrams	Uses structured text
Helps understanding	Helps implementation
Done before SRS	Results in SRS

Software Engineering Example

Online Examination System

---

1 Functional Requirements

(**WHAT the system does**)

- Student can **register and login**
- Student can **view available exams**
- Student can **attempt online exam**
- System can **auto-submit exam after time over**
- System can **calculate and display result**
- Admin can **add/update questions**
- Admin can **generate reports**

---

2 . Non-Functional Requirements

(**HOW the system works**)

- System should respond **within 2 seconds**
- System should be **secure** (no cheating, data protection)
- System should support **1000+ students simultaneously**
- System should be **available 24×7**
- System should be **user-friendly**

SRS: Online Examination System

1. Introduction

The Online Examination System is a web-based application that allows students to take exams online and enables administrators to manage exams efficiently.

---

2. Purpose

To automate the examination process and reduce manual effort while ensuring accuracy and security.

---

3. Scope

- Conduct online exams
- Automatic evaluation
- Result generation
- Exam management

---

4. Users

- Student
- Administrator

---

5. Functional Requirements

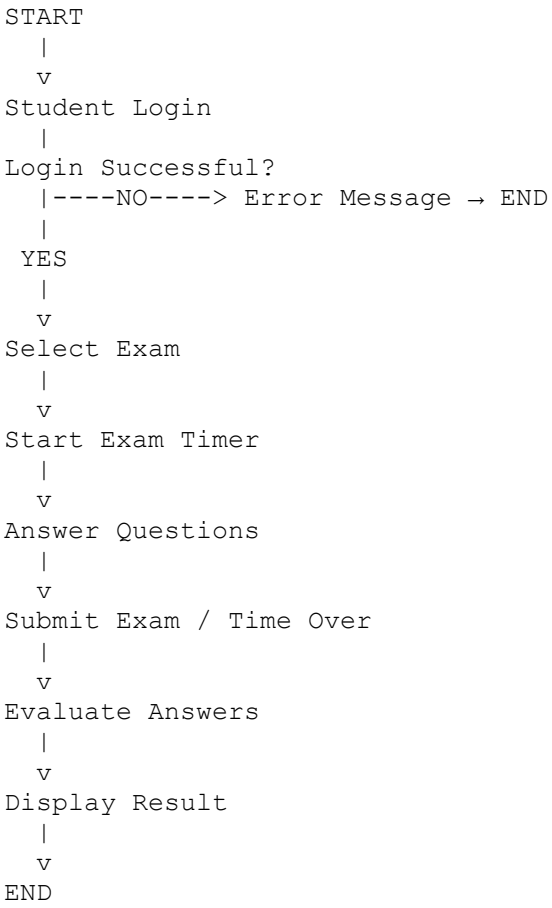
- FR1: The system shall allow students to **log in**.
- FR2: The system shall allow students to **attempt exams online**.
- FR3: The system shall **automatically submit** the exam after time expiration.
- FR4: The system shall **calculate results automatically**.
- FR5: The admin shall be able to **manage exams and questions**.

---

6. Non-Functional Requirements

- NFR1: The system shall provide **fast response time**.
- NFR2: The system shall be **secure and reliable**.
- NFR3: The system shall support **concurrent users**.
- NFR4: The system shall be **highly available**.

🔄 Flow Chart: Online Examination System



Online Examination System

(Complete Software Engineering Example)

---

1 Waterfall Model Mapping

Waterfall Model Phases applied to Online Exam System:

1. **Requirement Analysis**
   - Student login
   - Online exam
   - Result generation
2. **System Design**
   - Database design (Student, Exam, Result)
   - UI design (Login, Exam page)
3. **Implementation**
   - Coding of login, exam module, evaluation
4. **Testing**
   - Login testing

    - Exam submission testing
    - Result accuracy testing
5. **Deployment**
    - System deployed on server
6. **Maintenance**
    - Bug fixing
    - Question updates

☞ **Exam line:**

Waterfall model is suitable because requirements of online examination system are well defined.

---

2 Agile Model Mapping

Agile approach for Online Examination System:

- Sprint 1: Login & Registration
- Sprint 2: Exam Module
- Sprint 3: Auto Evaluation & Result
- Sprint 4: Reports & Security

☞ Continuous testing & feedback after each sprint.

---

3 Use Case Diagram (Textual Representation)

Actors:

- Student
- Admin

Use Cases:

- Login
- Attempt Exam
- View Result
- Manage Questions
- Generate Reports

SOFTWARE DESIGN CONCEPTS

Software design concepts are fundamental ideas used to structure and organize software properly.

---

1. Abstraction

Definition:
Abstraction means showing only important features and hiding unnecessary details.

Purpose: Reduce complexity and improve understanding.

Types:

- Data Abstraction → Hide data structure details
- Procedural Abstraction → Hide function logic

Example:
ATM machine → User enters PIN and withdraws money, but internal banking process is hidden.

---

2. Modularity

- **Definition:**
  Dividing a system into small independent modules. Each module performs a specific task.

Purpose: Easy testing, debugging, and maintenance.

Example:
Online Shopping System modules:

- Login Module
- Payment Module
- Order Module
- Delivery Module

Each module works independently.

---

3. Encapsulation (Information Hiding)

Definition:
Hiding internal data of a class/module and allowing access through methods.

Purpose: Security and data protection.

Example (Bank System):

```
class BankAccount:
   private balance
   public deposit()
   public withdraw()
```

User cannot directly change balance.

---

4. Separation of Concerns (SoC)

Definition:
Divide software so each part handles one responsibility.

Example (Web Application):

- UI Layer → Display
- Business Logic → Processing
- Database → Storage

---

5. Architecture

Definition:
Overall structure of software showing components and their interactions.

Common Architectures:

- Layered Architecture
- Client–Server
- MVC (Model-View-Controller)

Example:
Client sends request → Server processes → Database stores data.

---

6. Refinement (Stepwise Refinement)

Definition:
Start from high-level design and gradually move to detailed design.

Example:
Problem → Design modules → Write algorithms → Code

---

7. Functional Independence

A good module should have:

- **High Cohesion:** Focus on single task
- **Low Coupling:** Minimal dependency on other modules

Example:
Payment module should only handle payment, not delivery.

---

8. Design Patterns

Reusable solutions to common design problems.

Examples:

- Singleton → Only one object
- Factory → Object creation handled centrally
- Observer → Notification system (e.g., YouTube subscription)

SOFTWARE DESIGN PRINCIPLES

Design principles are rules to develop high-quality, maintainable, and scalable software.

---

1. SOLID Principles

S — Single Responsibility Principle (SRP)

Definition: One class → One responsibility.

Example:
Bad: `Student` class handles marks + fee + attendance
Good: Separate classes → `Marks`, `Fee`, `Attendance`

O — Open/Closed Principle (OCP)

Definition: Software should be open for extension but closed for modification.

Example:
Add new payment method (UPI) without modifying old code.

L — Liskov Substitution Principle (LSP)

Definition: Child class should behave like parent class.

Example:
`Bird → fly()`
If Penguin cannot fly → design wrong → break LSP.

I — Interface Segregation Principle (ISP)

Definition: Many small interfaces are better than one large interface.

Example:
Printer → Print + Scan + Fax
Small interfaces:

- Printable
- Scannable
- Faxable

D — Dependency Inversion Principle (DIP)

Definition: Depend on abstraction, not concrete class.

Example:
`Payment` depends on `PaymentInterface`, not `UPIPayment` directly.

---

2. DRY — Don’t Repeat Yourself

Avoid duplicate code.

Example:
Create one function `calculateTax()` instead of writing same code many times.

---

3. KISS — Keep It Simple, Stupid

Design should be simple and easy to understand.

Example:
Simple if-else better than complex nested logic.

---

4. YAGNI — You Aren’t Gonna Need ItZZl

Do not add features that are not required now.

---

5. High Cohesion

Module should perform one focused task.

Example:
Login module → only authentication.

---

6. Low Coupling

Modules should have minimum dependency.

Example:
Change in UI should not affect database.

---

7. Reusability

Design components so they can be reused.

Example:
Login system reused in multiple apps.

---

8. Maintainability

Software should be easy to modify and debug.

---

9. Scalability

System should handle increased load.

Example:
Cloud-based applications support many users.

SHORT DIFFERENCE (EXAM)

Design Concepts	Design Principles
Basic ideas of software structure	Rules for good design
Example: Abstraction, Modularity	Example: SOLID, DRY
Focus on system organization	Focus on quality & maintainability

Short Exam Summary

Design Concepts

- Abstraction
- Modularity
- Encapsulation
- Separation of Concerns
- Functional Independence
- Refinement
- Design Patterns
- Architecture

Design Principles

- SOLID
- DRY
- KISS
- YAGNI
- High Cohesion, Low Coupling
- Law of Demeter

---

1. Architectural Design

Architectural design defines the **overall structure of the system** (components + communication).

Common Architecture Styles

1. Layered Architecture

System divided into layers.

Layers

1. Presentation Layer (UI / Mobile App)
2. Application Layer (Logic)
3. Data Layer (Database)

Example – Smart Home
User App → Control Logic → Device Database

Advantages

- Easy maintenance
- Clear structure

---

2. Client–Server Architecture

- Client sends request
- Server processes and responds

Example
Mobile App (Client) → Cloud Server → Smart Devices

---

3. MVC (Model–View–Controller)

- Model → Data
- View → UI
- Controller → Logic

Example
Model: Device status

View: Mobile screen
Controller: ON/OFF logic

---

4. Microservices Architecture

System divided into small independent services.

Example:

- Lighting Service
- Temperature Service
- Security Service

---

2. UML Diagrams

UML = **Unified Modeling Language**
Used to visually represent system design.

2.3 Sequence Diagram

Shows **step-by-step interaction over time**.

Smart Home: Turn ON Light

1. User presses ON button
2. App sends request to Server
3. Server sends command to Device
4. Device turns ON
5. Device sends status back
6. App shows "Light ON"

Flow:

User → App → Server → Device → Server → App → User

2.4 Activity Diagram

Shows **workflow / process flow**.

Smart Home: Control Device

Start
↓
User Login
↓
Select Device
↓
Choose ON/OFF
↓
Send Command
↓
Device Responds
↓
Show Status
↓
End

Exam Ready Definitions

Architectural Design → Structure of system components and communication.

Use Case Diagram → Interaction between user and system.

Class Diagram → Static structure (classes + attributes + methods).

Sequence Diagram → Time-based interaction between objects.

Activity Diagram → Workflow of system.

UML DIAGRAMS (DRAWING FORMAT FOR EXAM)

1. Use Case Diagram – Drawing Steps

Steps to Draw

1. Draw **system boundary box**
2. Draw **actors (stick man)** outside box
3. Draw **ovals (use cases)** inside box
4. Connect actor → use case using line

Smart Home Example

Actors:

- User
- Admin

Use Cases inside box:

- Login
- View Device Status
- Turn ON/OFF Device
- Set Temperature
- Receive Alert
- Manage Devices (Admin)

Exam Tip:
Actor left side, system box center, use cases inside.

Use Case Diagram – BookMyShow

Actors

1. **User / Customer**
2. **Admin**
3. **Payment Gateway** (External System)

Main Use Cases

For User

- Register / Login
- Search Movie / Event
- View Details (Timing, Seat, Price)
- Select Seats
- Book Ticket
- Make Payment
- Download / Print Ticket
- Cancel Ticket

For Admin

- Add Movie / Event
- Update Movie / Event
- Delete Movie / Event
- Manage Shows & Timing

- View Booking Reports

How to Draw (Step-by-Step)

1. Draw **System Boundary Box** → Write **BookMyShow System** at top
2. Draw **Actors (stick figures)** outside box:
    - Left → User
    - Right → Admin
    - Side → Payment Gateway
3. Draw **Ovals (Use Cases)** inside box
4. Connect actors with related use cases using straight lines
5. Add <<**include**>> between:
    - *Book Ticket* → *Make Payment*
6. Add <<**extend**>> between:
    - *Book Ticket* → *Cancel Ticket* (optional, if cancellation allowed after booking)
    -

```
User ---- Register/Login
User ---- Search Movie/Event
User ---- View Details
User ---- Select Seats
User ---- Book Ticket ---- <<include>> ---- Make Payment ---- Payment Gateway
User ---- Download Ticket
User ---- Cancel Ticket

Admin ---- Add Movie/Event
Admin ---- Update Movie/Event
Admin ---- Delete Movie/Event
Admin ---- Manage Shows
Admin ---- View Reports
```

Shows **interaction between user and system**.

Smart Home Use Cases

Actors

- User
- Admin

Use Cases

- Login
- View Device Status
- Turn ON/OFF Device
- Set Temperature

- Receive Alert

Text Diagram Representation

User → Login
User → Turn ON/OFF Light
User → View Temperature
Admin → Manage Devices

2.Class Diagram – Drawing Steps

Steps to Draw

1. Draw **rectangle with 3 parts**
    - Class Name (Top)
    - Attributes (Middle)
    - Methods (Bottom)
2. Show **relationships**
    - Association → simple line
    - Inheritance → arrow
    - Composition → filled diamond

2.1 Class Diagram

Shows **classes, attributes, methods, relationships**.

2.1 Smart Home Classes

Class: User

Attributes: userID, name
Methods: login(), controlDevice()

Class: Device

Attributes: deviceID, status
Methods: turnOn(), turnOff()

Class: Sensor

Attributes: sensorID, value
Methods: readValue()

Relationship

User → controls → Device
Device → connected to → Sensor

Smart Home Classes

User

- userID
- name

- login()
- controlDevice()

Device

- deviceID
- status

- turnOn()
- turnOff()

Sensor

- sensorID
- value

- readValue()

Relations:
User —— controls —— Device
Device ◆— connected — Sensor

+----------------------+

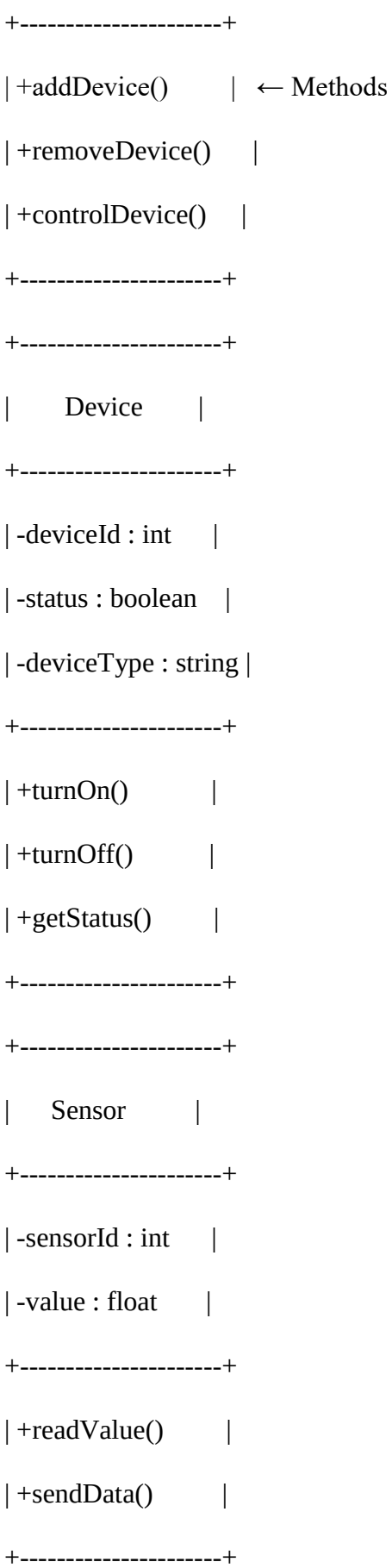
| SmartHome | ← Class Name

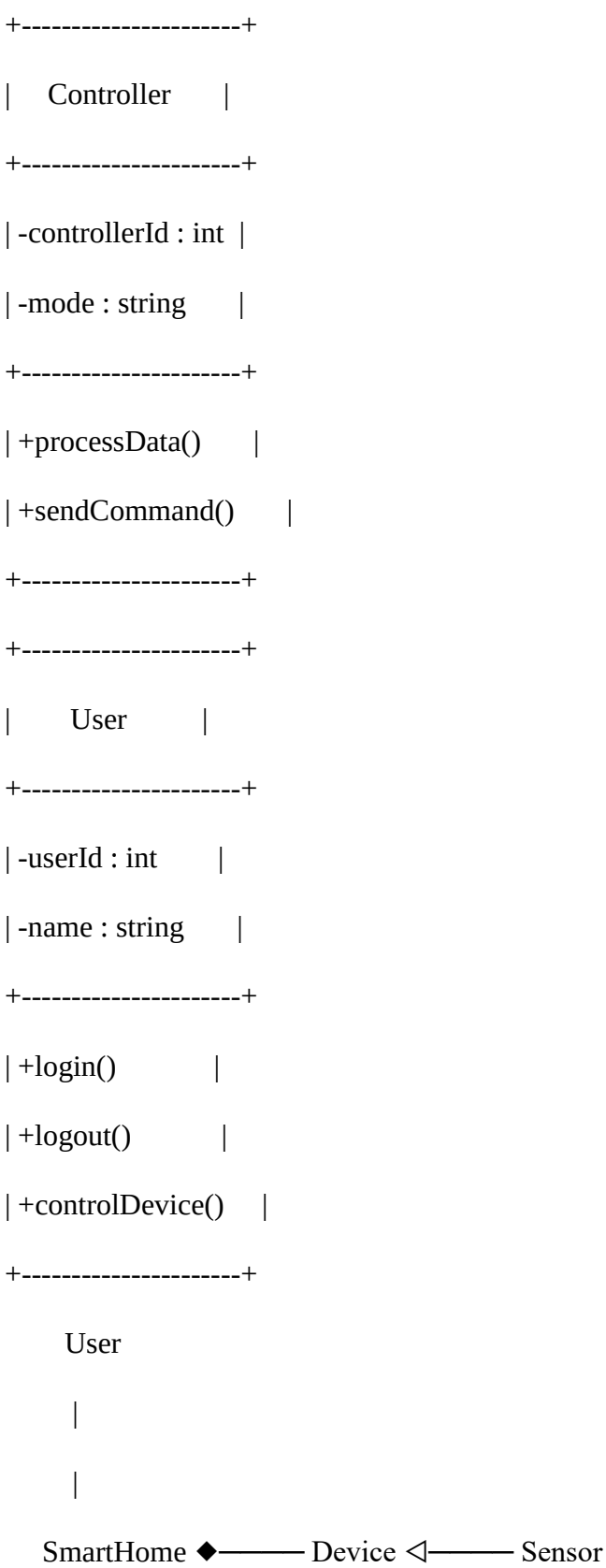
+----------------------+

| -homeId : int | ← Attributes

| -ownerName : string |

| -location : string |


+addDevice()
← Methods
+removeDevice()
+controlDevice()
Device
-deviceId : int
-status : boolean
-deviceType : string
+turnOn()
+turnOff()
+getStatus()
Sensor
-sensorId : int
-value : float
+readValue()
+sendData()


+----------------------+
| Controller |
+----------------------+
| -controllerId : int |
| -mode : string |
+----------------------+
| +processData() |
| +sendCommand() |
+----------------------+
+----------------------+
| User |
+----------------------+
| -userId : int |
| -name : string |
+----------------------+
| +login() |
| +logout() |
| +controlDevice() |
+----------------------+
User
|
|
SmartHome ◆——— Device ◁——— Sensor
|
|

Controller ———— Device

---

3. Sequence Diagram – Drawing Steps

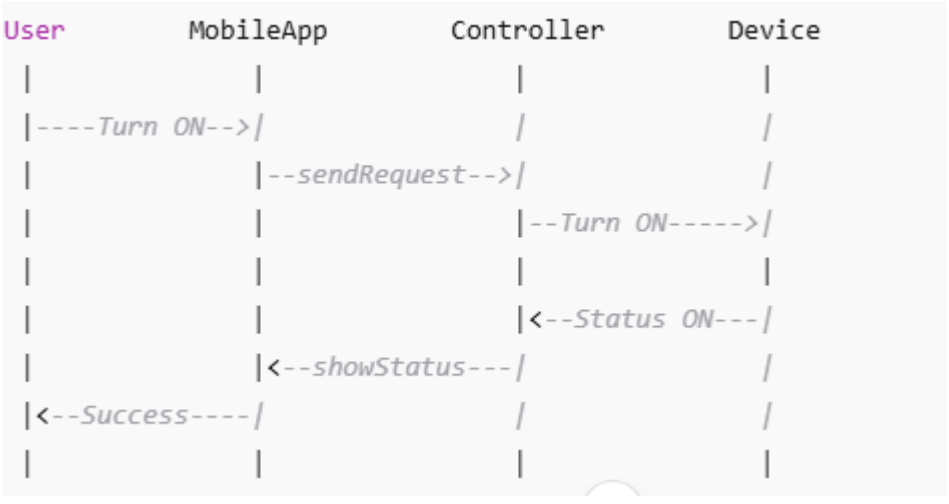
Steps

1. Draw **actors & objects horizontally**
2. Draw **vertical dotted lifeline**
3. Draw arrows (messages) top → bottom
4. Show return message (dashed arrow)

Smart Home – Turn ON Light

Objects:
User | App | Server | Device

Flow:
User → App : Press ON
App → Server : Send Request
Server → Device : Turn ON
Device → Server : Status ON
Server → App : Success
App → User : Light ON



---

4. Activity Diagram – Drawing Steps

Symbols

- ● Start

- □ Process
- ◇ Decision
- → Flow line
- ◎ End

Smart Home – Control Device

Start ●
↓
Login □
↓
Select Device □
↓
ON or OFF ◇
↓
Send Command □
↓
Show Status □
↓
End ◎